

1. Práctica 1: Persistencia - Memoria FLASH

1.1. Objetivos

- Entender el concepto de persistencia y para qué datos sirve.
- Entender el funcionamiento de la memoria **FLASH**.

1.2. Fundamentos teóricos

Todos damos por sentado que, en un ordenador al uso, podemos guardar datos que se mantendrán tras el reinicio. Sin embargo, en los dispositivos empujados, muchas veces no ocurre lo mismo: Los programas se ejecutan en RAM y, al terminar, toda su información se ha borrado. Si, por ejemplo, queremos tener cierta configuración de usuario (como pueda ser la temperatura en un termostato, por ejemplo) hay que asegurarse de alguna manera que se mantiene.

Una forma de hacerlo es mediante memoria persistente **FLASH**. Este tipo de memoria tiene una peculiaridad, y es que, si bien consigue guardar datos incluso sin corriente, al contrario que la RAM, se desgasta con el uso. Adicionalmente, sólo se pueden escribir “0” y borrar a “1” (lo que se conoce como un ciclo de escritura-borrado). Por tanto, cada vez que la queremos reprogramar, desgastamos su estructura. El procesador **STM32F723I** tiene una garantía mínima de un aguante de 10000 ciclos de este proceso. Aún así, se recomienda encarecidamente limitar estos números lo máximo posible. Para esta práctica, en concreto, intenta limitar el número de borrados de la **FLASH**. Si tienes cualquier duda, consulta a tu profesor.

En cuanto a las lecturas, no supone ningún problema, y de hecho nuestro programa se guarda en la **FLASH**, para que se ejecute desde ahí las veces que sea necesario, y sus valores volátiles en la **RAM**, evitando así ese desgaste continuo.

1.2.1. Organización de la flash

En nuestro procesador, la memoria **FLASH** se organiza en *sectores* ([rm0431 3.3.1](#)). Podemos escribir byte a byte en cada sector, sin embargo para resetearlo, deberemos resetear los sectores enteros. Este diseño hace que se desgaste menos la memoria, al haber generalmente un programa en únicamente unos pocos sectores, y no necesitar borrar el resto. Es lo mismo que ocurre con los discos SSD, que pueden ir repartiendo el desgaste entre sus sectores para durar más.

1.3. Desarrollo de la práctica

1.3.1. Bucle de control principal

En esta práctica vamos a hacer que la última coordenada pulsada en la pantalla táctil se guarde de manera persistente, de tal manera que, tras reiniciar (o incluso desconectar de la corriente) nuestro procesador, la podamos volver a ver tras un nuevo arranque. Condicionaremos el guardado con el pulsado de un botón, para evitar que se desgaste en exceso. El esquema general es el siguiente:

```

1  int main(void)
2  {
3      // Inicializaciones
4
5      if (UserButton_IsPressed()) {
6          // Borrar memoria persistente
7      }
8
9      // Attempt reading saved coordinates
10     if (Flash_ReadTouch(&last_x, &last_y)) {
11         // Si se encuentran datos en la memoria, cargarlos
12         // dibujando un cuadrado verde en la posición guardada
13     } else {
14         // Si no se encuentran datos, dibujar un cuadrado amarillo
15     }
16
17     while (1) {
18         // Lógica de actualización de la pantalla
19         // Save new touch coordinates into Flash
20         if (UserButton_IsPressed()) {
21             // Guardar coordenada en memoria persistente
22         }
23     }
24 }

```

1.3.2. Trabajando con la FLASH

En primer lugar, necesitamos completar nuestro `bsp.h` con el código necesario para exponer los registros de la `FLASH` (`rm0431 3`).

```

1  #define FLASH_BASE_ADDR      0x40023C00UL
2
3  typedef struct {
4      volatile uint32_t ACR;      // 0x00
5      volatile uint32_t KEYR;     // 0x04
6      volatile uint32_t OPTKEYR;  // 0x08
7      volatile uint32_t SR;       // 0x0C
8      volatile uint32_t CR;       // 0x10
9      volatile uint32_t OPTCR;    // 0x14
10     volatile uint32_t OPTCR1;    // 0x18
11     volatile uint32_t OPTCR2;    // 0x1C
12 } FLASH_TypeDef;
13
14 #define FLASH                ((FLASH_TypeDef *) FLASH_BASE_ADDR)

```

Utilizar la `FLASH` no es tan fácil como escribir directamente en sus registros. Ya hemos mencionado en un par de ocasiones que la memoria se desgasta. No es de extrañar, por tanto, que haya un cierto nivel de protección para trabajar con ella. Aunque es accesible directamente para lectura (podemos leer desde la dirección `0x08000000` en adelante, y estaremos leyendo desde la `FLASH` (`rm0431 1.6.2`)), la escritura está protegida. Adicionalmente a evitar el desgaste, evitamos escribir accidentalmente en una región que suele contener únicamente código y constantes (Ver el archivo de linkado `STM32F723IEKX_FLASH.ld` para comprobarlo).

La `FLASH` está siempre activa, así que nos ahorraremos activar periféricos o relojes. Pero lo primero, antes de trabajar con ella, si queremos realizar algún cambio, es desbloquear el acceso que previene escrituras accidentales. Para ello hay que escribir una secuencia específica de datos

en el registro **KEYR** ([rm0431 3.7.2](#)) tras comprobar que no esté bloqueada. Si por el contrario queremos bloquearla, esto se hace de manera más sencilla configurando el bit correspondiente en **CR** ([rm0431 3.7.5](#))

```
1  /* Flash Keys */
2  #define FLASH_KEY1      0x45670123U
3  #define FLASH_KEY2      0xCDEF89ABU
4  static void inline Flash_Unlock(void) {
5      // ... completar aplicar secuencia de desbloqueo en KEYR
6  }
7
8  static void inline Flash_Lock(void) {
9      // ... completar bloquear en registro CR
10 }
```

Ciertas operaciones con la **FLASH** requerirán esperar a que el periférico deje de estar ocupado (no deja de ser una interfaz con la memoria, que no necesariamente opera a la misma velocidad que el procesador). Incluimos también la función que realiza una espera activa a que se libere el bit correspondiente en **SR**: ([rm0431 3.7.4](#)).

```
1  static void inline Flash_WaitNotBusy(void) {
2      while (FLASH->SR & (1U << 16)); // Wait for BSY bit
3  }
```

Con estas funciones estamos listos para, en primer lugar, borrar un sector. Recordemos que es necesario borrar (poner todo a “1”) antes de poder escribir (que grabará sólo los “0” correspondientes). Esto es debido a que la tecnología subyacente sólo permite este tipo de trabajo. Programamos la función de borrado desde el registro **CR** ([rm0431 3.7.5](#)). Hay previamente que solicitar el desbloqueo de flash, y con posterioridad bloquearla para evitar escrituras accidentales.

```
1  static void inline Flash_EraseSector(uint8_t sector) {
2      Flash_Unlock();
3      Flash_WaitNotBusy();
4
5      // ... completar borrar los bits de sector SNB
6      // ... completar programar el sector que queremos borrar
7      // ... completar marcar el bit de borrado SER
8      // ... completar iniciar el borrado con el bit STRT
9
10     Flash_WaitNotBusy();
11     // ... completar desactivar el borrado SER
12     Flash_Lock();
13 }
```

NOTA: Ten cuidado con los sectores que borras, ya que en los primeros (0-3) se suele guardar el código del programa. Si lo borras, lo más seguro es que la aplicación se bloquee, ten en cuenta la estructura de sectores de la Figura 1.

Ahora, si queremos escribir, el proceso será ligeramente distinto. Por el momento, ya que no vamos a escribir demasiada información, vamos a ir byte a byte, pero la función se puede adaptar para escribir posteriormente en bloques, sin necesidad de bloquear y desbloquear con cada transferencia de información. De nuevo trabajamos con el registro **CR** para, en este caso, habilitar la escritura. Observemos también la secuencia para escribir: lo vamos a hacer directamente escribiendo en la dirección de memoria requerida (de la misma forma que lo hacíamos en las prácticas iniciales). El controlador de la **FLASH** recibirá el dato y, tras ello, lo escribirá.

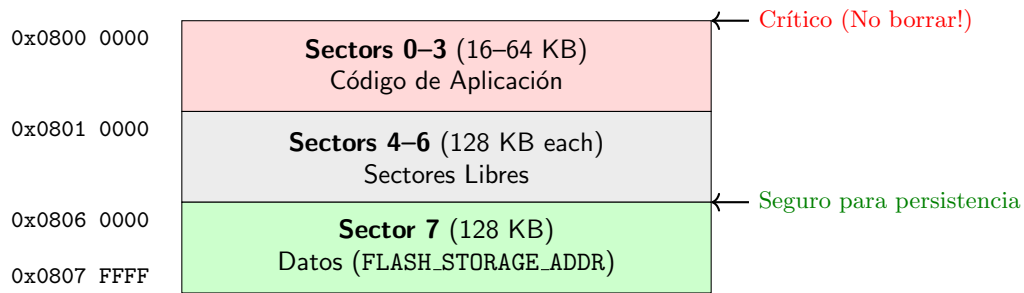


Figura 1: Organización de los sectores de Flash

Como puede tardar varios ciclos, forzamos mediante primitivas el vaciado de los *store buffers*, y hacemos una espera a que el periférico nos confirme que ha terminado:

```

1 static void inline Flash_WriteByte(uint32_t address, uint8_t data) {
2     Flash_Unlock();
3
4     // ... completar Ajustar el tamaño de transferencia PSize a '00'
5     // ... habilitar la programación de FLASH con el bit PG a 1
6
7     *(volatile uint8_t *)address = data;
8     __asm volatile ("dsb 0xF" ::: "memory"); // Pipeline flush
9     Flash_WaitNotBusy();                     // Wait for BSY bit to clear
10
11     // ... deshabilitar la programación de FLASH con el bit PG a 0
12
13     Flash_Lock();
14 }

```

De la lectura no hay que preocuparse, pues podremos hacer lecturas directas a las direcciones de memoria atacadas.

1.3.3. Creando las estructuras para la persistencia

Con esto ya tenemos la **FLASH** preparada para trabajar con ella con todas las funciones auxiliares. Ahora, lo que nos falta es crear un nuevo conjunto de funciones que nos permitan escribir una estructura de datos (que queramos en este caso guardar) utilizándolas. Creamos un nuevo archivo (por ejemplo *persistence.h*) con el siguiente contenido:

```

1 /*
2  * persistence.h
3  *
4  * Created on: Aug 27, 2026
5  * Author: dani
6  */
7
8 #ifndef PERSISTENCE_H_
9 #define PERSISTENCE_H_
10
11 #define FLASH_SECTOR_NUM    7U                // Sector 7 (safe upper Flash
12     sector)
13 #define FLASH_STORAGE_ADDR  0x08060000U      // Base address of Sector 7
14 #define FLASH_MAGIC_KEY     0xCAFEBAEU      // Marker to verify stored data
15     validity

```

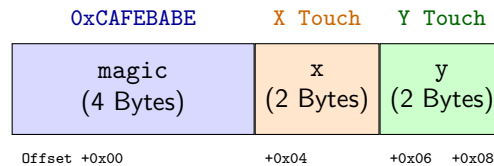


Figura 2: Organización de la estructura de guardado en memoria

```

15 typedef struct {
16     uint32_t magic;
17     uint16_t x;
18     uint16_t y;
19 } FlashTouchData;
20
21 /* Erases Sector 7 and writes the magic key + touch coordinates */
22 void Flash_SaveTouch(uint16_t x, uint16_t y) {
23     FlashTouchData data = {
24         .magic = FLASH_MAGIC_KEY,
25         .x = x,
26         .y = y
27     };
28
29     Flash_EraseSector(FLASH_SECTOR_NUM);
30
31     uint8_t *pData = (uint8_t *)&data;
32     for (uint32_t i = 0; i < sizeof(FlashTouchData); i++) {
33         Flash_WriteByte(FLASH_STORAGE_ADDR + i, pData[i]);
34     }
35 }
36 #endif /* PERSISTENCE_H_ */

```

Hay que fijarse en un detalle interesante: Aunque queremos guardar un conjunto de coordenadas (x, y), nuestra estructura (Figura 2) contiene un tercer campo llamado *magic* con un valor fijo. El guardar un valor como este es muy importante para poder saber si disponemos de valores válidos en las direcciones de memoria que consultemos. De lo contrario, estaríamos cargando valores aleatorios almacenados previamente por otros programas o usuarios. Programa ahora la función simétrica para realizar la lectura:

```

1 /* Reads coordinates from Flash if a valid magic key is present */
2 uint8_t Flash_ReadTouch(uint16_t *x, uint16_t *y) {
3     // ... completar conseguir un puntero de tipo FlashTouchData
4     // ... a la dirección base para poder leerla
5
6     // ... si encontramos la MAGIC KEY, devolver 1
7     // ... además de rellenar los valores x, y
8     // ... de lo contrario, devolver 0 (Flash vacía)
9 }

```

Y... casi listo! Sólo nos falta completar el código de `main.c` anteriormente presentado, llamando ahora sí a estas funciones auxiliares que hemos creado para guardar y leer la **FLASH**. Comprueba que, tras guardar un punto, e incluso desconectando la placa de la corriente, se lee el dato guardado en memoria.

1.4. Para hacer más

Existe otra forma de persistencia en memoria mediante la **BKPSRAM** o memoria de *backup*. Es una zona de memoria que mantiene sus contenidos incluso tras resetear el procesador (eso sí, si se pierde la corriente, se borrarán igualmente). No es tan persistente como la **FLASH**, pero no sufre de desgaste. Investiga cómo podrías hacer la persistencia con esta memoria ([rm0431 4.1.3](#)).

CURIOSIDAD: Los antiguos juegos de videoconsolas como la *Game Boy* funcionaban con este tipo de memoria. Los cartuchos incluían una pila para mantener las partidas guardadas. Si se acababa la pila, se perdían los datos. Duraban unos cuantos años, pero debido a esa tecnología se han perdido muchos recuerdos.